

Laboratorio di Basi di Dati: *Progetto "Delivery"*

Gruppo di lavoro:

Matricola	Nome	Cognome	Contributo al progetto
295438	Alessia	De Dominicis	- Scelte progettuali e risoluzione delle ambiguità (entrambi) - Formalizzazione dei vincoli non esprimibili nel modello ER (entrambi) - Ristrutturazione ed ottimizzazione del modello ER - Script di popolamento (insert.sql) - Implementazione vincoli - Funzionalità 1, 3, 5, 7, 9, 11, 13
271770	Riccardo	D'Aviero	- Scelte progettuali e risoluzione delle ambiguità (entrambi) - Modello ER - Formalizzazione dei vincoli non esprimibili nel modello ER (entrambi) - Traduzione del modello ER nel modello relazionale - Implementazione del modello relazionale (create.sql) - Funzionalità 2, 4, 6, 8, 10, 12, 14

Data di consegna del progetto: 02/09/2026

Indice

1. [Formalizzazione ed analisi dei requisiti](#)
 - 1.1 [Specifiche del progetto](#)
 - 1.2 [Scelte Progettuali e Risoluzione delle Ambiguità](#)
2. [Progettazione concettuale](#)
 - 2.1 [Modello Entità-Relazione](#)
 - 2.2 [Formalizzazione dei vincoli non esprimibili nel modello ER](#)
3. [Progettazione logica](#)
 - 3.1 [Ristrutturazione ed ottimizzazione del modello ER](#)
 - 3.2 [Traduzione del modello ER nel modello relazionale](#)
4. [Progettazione fisica](#)
 - 4.1 [Implementazione del modello relazionale](#)
 - 4.2 [Implementazione dei vincoli](#)

4.3 Implementazione funzionalità richieste

1. Formalizzazione ed analisi dei requisiti

1.1 Specifiche del progetto

Descrizione del dominio

Il database *Delivery* supporta una generica attività di ristorazione che offre servizio di consegna a domicilio.

L'attività disporrà di un *menu* composto da *prodotti* ognuno dotato almeno di un nome, una descrizione, un prezzo e, opzionalmente, una (o più) immagini. Internamente (cioè in modo non visibile ai clienti), ogni prodotto avrà anche associati un tempo di preparazione, una lista di ingredienti (con quantità) e opzionalmente la descrizione testuale della procedura di preparazione. Deve essere prevista anche la possibilità di scegliere tra differenti *caratteristiche* del prodotto, ognuna dotata di nome, descrizione (opzionale) e di differenza prezzo (rispetto al prezzo base del prodotto). Alcune caratteristiche potranno essere di *default* (quindi pre-selezionate). Infine, si potranno creare *gruppi di mutua esclusione* tra sottoinsiemi delle caratteristiche (in modo che solo una delle caratteristiche nel gruppo possa essere selezionata). *Esempio: il prodotto "caffè" potrebbe costare 1 euro, e avere come caratteristiche "senza zucchero" (-5 centesimi), "zuccherato" (default) e "molto zuccherato", raggruppate in un gruppo di mutua esclusione chiamato "zucchero", oltre a "con panna" (+50 centesimi) e "freddo" (+1 euro), liberamente selezionabili (non raggruppate).*

I *clienti* potranno registrarsi liberamente nel sistema, ma si noti che i dati di un cliente dovranno necessariamente comprendere contatti telefonici e indirizzo, visto che parliamo di consegna a domicilio.

I clienti potranno selezionare uno o più prodotti dal menu, comprensivi di caratteristiche se presenti, creando un *ordine* a loro nome. L'utente potrà quindi confermare o annullare l'ordine. In caso di conferma, l'utente potrà anche selezionare un orario specifico per la consegna.

L'applicazione avrà anche altre due tipologie di utenti: il *proprietario*, che supponiamo pre-caricato nel sistema all'atto della sua installazione, e il *personale*. Il primo potrà monitorare gli ordini passati, in preparazione ed evasi, potrà comporre il menu inserendo o modificando tutte le informazioni inerenti i prodotti e potrà infine registrare membri del personale.

I membri del personale vedranno la lista degli ordini correnti, che potranno avere cinque stati: *inserito, in preparazione, pronto, in consegna e consegnato*. Il personale potrà cambiare lo stato dell'ordine in qualsiasi momento, ma solo seguendo l'ordine progressivo (non si potrà riportare un ordine in consegna nello stato di preparazione). Il sistema dovrà tener traccia dell'effettivo membro del personale che effettua ciascun cambio di stato su un ordine.

Operazioni da realizzare

Di seguito sono illustrate schematicamente le operazioni previste sulla base di dati, ciascuna da realizzare tramite una query (o, se necessario, tramite più query, *opzionalmente* racchiuse in una *stored procedure*). Ovviamente, ogni ulteriore raffinamento o arricchimento di queste specifiche aumenterà il valore del progetto.

1. Generazione del menu (*lista dei prodotti con tutte le informazioni visibili al cliente, possibilmente anche le relative caratteristiche con la differenza di prezzo*).
2. Eliminazione di una caratteristica associata a un prodotto.
3. Inserimento di un prodotto in un ordine, comprensivo delle sue eventuali caratteristiche.
4. Calcolo del tempo stimato di consegna e del prezzo totale di un ordine (*suggerimento: potete provare a usare una sotto-query per calcolare la differenza cumulativa di prezzo derivante dalle caratteristiche selezionate e poi sommarla al prezzo base*).
5. Lista degli ordini non ancora messi in preparazione dopo più di un'ora dall'inserimento (*suggerimento: è quindi necessario prevedere degli opportuni timestamp da affiancare agli stati*).
6. Calcolo del tempo medio di consegna, cioè di passaggio tra lo stato *in consegna* in quello *consegnato*, per ciascun membro del personale addetto alla consegna (*supponiamo che chi consegna sia colui il quale imposta lo stato su consegnato*).
7. Classifica di gradimento dei prodotti (*quali prodotti compaiono più comunemente negli ordini?*).
8. Calcolo dell'incasso giornaliero.
9. Prospetto del consumo di ingredienti in un anno (*quantità di ciascun ingrediente consumata in un certo anno*).
10. Estrazione dei prodotti preferiti da un cliente (*cioè i prodotti più ordinati da quel cliente, magari escludendo a priori quelli ordinati solo un paio di volte...*).
11. Conteggio degli ordini attivi (non in stato *consegnato*) divisi per il loro stato di avanzamento.
12. Conteggio degli ordini smaltiti (consegnati) in uno specifico giorno.
13. Lista dei membri del personale che hanno lavorato a un particolare ordine.
14. Aggiornamento dello stato di un ordine.

1.2 Scelte Progettuali e Risoluzione delle Ambiguità

Durante l'analisi della specifica, sono emerse alcune ambiguità o necessità di raffinamento del dominio, che sono state risolte come segue:

- **Gestione dello storico prezzi:** Lo storico ordini dovrebbe riportare come prezzo totale di un ordine effettuato il prezzo effettivo pagato dal cliente al momento della conferma, e non dovrebbe ricalcolarlo in base al prezzo attuale dei prodotti: il proprietario potrebbe modificare i prezzi di un prodotto e ciò poi modificherebbe retroattivamente il prezzo totale anche di tutti gli ordini passati che contenevano quel prodotto. Abbiamo, quindi, scelto di storicizzare il prezzo con gli attributi `prezzo_base_al_momento` e `diff_prezzo_al_momento` legati direttamente alla fase di composizione dell'ordine.
- **Identificazione univoca degli ordini:** Inizialmente si era valutato di identificare un ordine tramite la data e ora di inserimento combinata all'identificativo del cliente. Tuttavia potrebbe crearsi la situazione in cui un utente effettua due ordini nello stesso "istante", magari potrebbe effettuare due

ordini nell'arco di un minuto e il sistema potrebbe non aggiornare l'orario, o per problemi di rete. Per prevenire eventuali conflitti, abbiamo scelto di modellare l'ordine come entità forte, identificata da un codice di ordine univoco, che potrebbe essere banalmente il codice sulla ricevuta.

- **Tracciamento dei cambi di stato:** Per soddisfare il requisito di tener traccia di quale membro del personale effettua ciascun cambio di stato, si è introdotta l'entità debole Storico Stato Ordine. La chiave parziale scelta è l'attributo stato: questa scelta è giustificata dal fatto che la specifica impone un avanzamento rigorosamente progressivo e non ciclico degli stati ("non si potrà riportare un ordine in consegna nello stato di preparazione"), garantendo quindi che uno stesso stato non si ripeta mai due volte per lo stesso ordine. Inoltre, il tracciamento del membro del personale responsabile di ogni cambio di stato riguarda solo le transizioni da "inserito" in poi perché i membri del personale non inseriscono gli ordini, bensì è il cliente. La creazione dell'ordine viene quindi tracciata separatamente con un timestamp di inserimento sull'ordine stesso (**data_inserimento**), invece dello storico degli stati.
- **Caratteristiche non raggruppate:** La specifica prevede esplicitamente caratteristiche "liberamente selezionabili (non raggruppate)" accanto a quelle organizzate in gruppi di mutua esclusione. Abbiamo quindi modellato il legame Prodotto–Caratteristica come relazione diretta e obbligatoria, mentre l'appartenenza a un Gruppo di mutua esclusione è stata modellata come opzionale per la singola caratteristica.
- **Numero minimo di caratteristiche per gruppo:** Abbiamo deciso di vincolare la relazione Include tra Gruppo Caratteristica e Caratteristica con cardinalità (2,n) sul lato Gruppo Caratteristica. Abbiamo deciso ciò perché un gruppo di mutua esclusione ha senso solo se permette di scegliere tra almeno due opzioni.
- **Dati obbligatori del cliente:** La specifica impone che un cliente debba necessariamente inserire numero di telefono e indirizzo per la consegna, questo vincolo vale però solo per i clienti e non per i membri del personale e il proprietario che non hanno bisogno di inserire questi dati. Abbiamo modellato questi dati come attributi specifici di Cliente e non di Utente.
- **Relazione proprietario–personale:** Abbiamo scelto di non modellare esplicitamente un legame tra Proprietario e i membri del Personale da lui registrati: la relazione tra i membri del personale e il proprietario sarebbe sempre la stessa, essendoci (da specifica) un unico proprietario è quindi scontato che l'inserimento lo effettui sempre lui. Se avessimo invece deciso di tenere traccia di più proprietari, avrebbe avuto senso introdurre questa relazione per tenere traccia di quale proprietario avesse inserito un determinato membro del personale.
- **Gestione immagini multiple:** Per associare un'immagine ad un prodotto, avevamo pensato di includere un semplice attributo **percorso_immagine** a Prodotto, ma la specifica dice che un prodotto potrebbe avere anche più di un'immagine associata ad esso. Abbiamo deciso quindi di creare l'entità debole Immagine che contiene **percorso_file** e **is_copertina**. Per quanto il percorso del file potrebbe univocamente identificare un'immagine, abbiamo deciso di non modellarla come entità forte perché non avrebbe senso memorizzare immagini se esse non sono associate ad un prodotto.
- **Calcolo del tempo stimato di consegna:** Per quanto riguarda il calcolo del tempo di preparazione di un ordine, assumiamo che la cucina del locale possa preparare più prodotti in contemporanea. Quindi, il tempo di preparazione dell'ordine sarà il tempo di preparazione del prodotto più lento, a

cui andrà aggiunto un tempo fisso stimato per il tragitto di consegna, e non la somma dei tempi dei singoli prodotti.

2. Progettazione concettuale

2.1 Modello Entità-Relazione

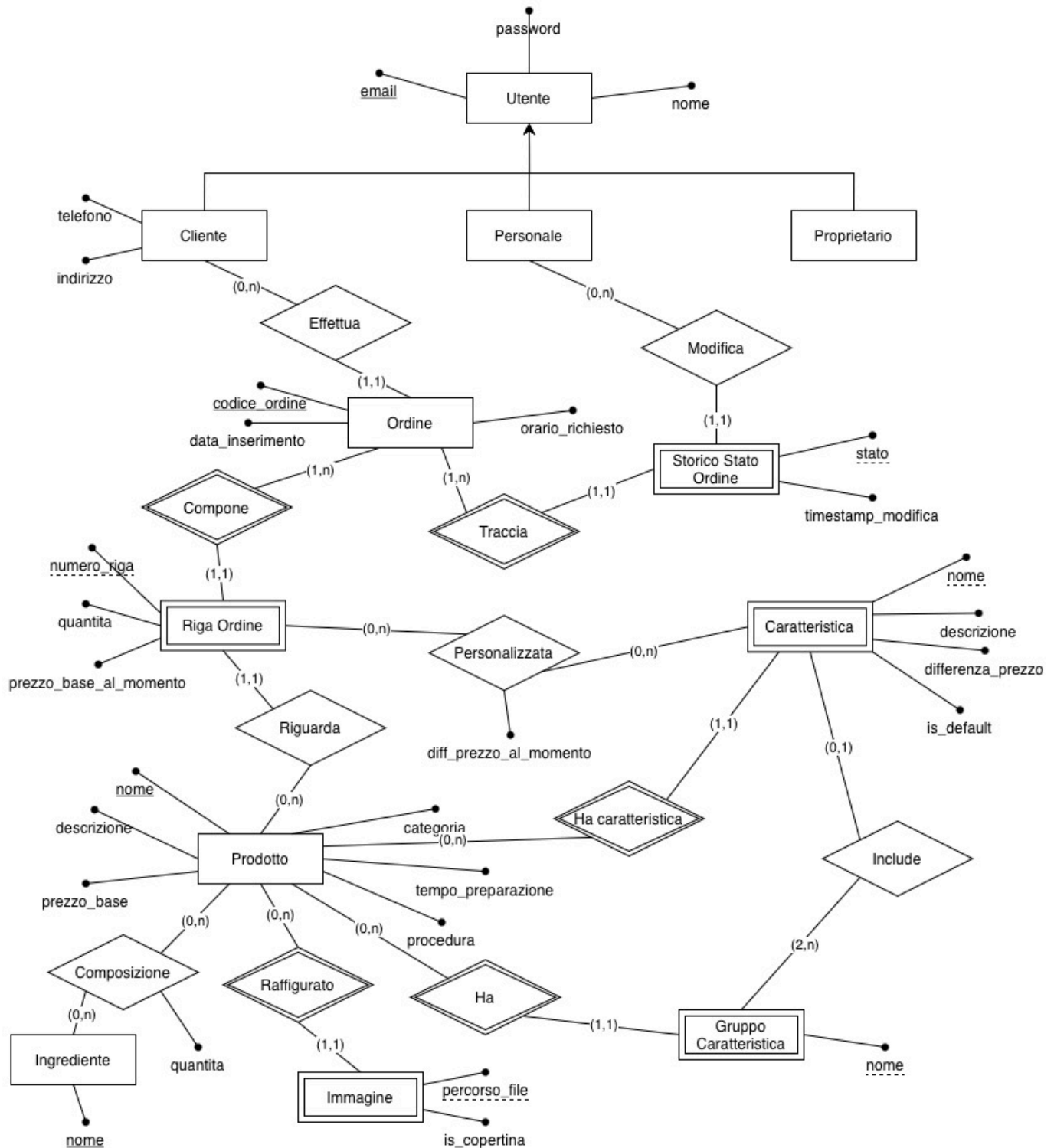


Diagramma realizzato con draw.io

Descrizione entità

- **Utente:** Entità padre di Cliente, Personale, Proprietario, rappresenta chiunque interagisca col sistema. Identificato univocamente dalla propria email.
- **Prodotto:** Elemento del menu acquistabile dal cliente. Possiede un prezzo base e una descrizione pubblici e dettagli interni per la preparazione (tempo, procedura, ingredienti). Identificato univocamente dal suo nome (realisticamente un ristorante non avrà dei prodotti chiamati allo stesso modo).
- **Ingrediente:** Materia prima necessaria per la composizione di un prodotto. Identificato univocamente dal suo nome.
- **Caratteristica:** Variazione o opzione applicabile a un prodotto. Può comportare un sovrapprezzo o uno sconto rispetto al prezzo base. Una caratteristica può essere di default (**is_default**). Identificata univocamente dal suo nome e dal prodotto che la contiene.
- **Gruppo Caratteristica:** Insieme di caratteristiche per le quali vale la regola della mutua esclusione (all'interno del gruppo, è possibile selezionare al massimo una singola opzione per prodotto). Identificato univocamente dal suo nome e dal prodotto che lo contiene.
- **Ordine:** Richiesta di acquisto effettuata da un cliente. Comprende l'orario di consegna richiesto e segue un ciclo di vita a stati (inserito, in preparazione, pronto, in consegna, consegnato). Identificato univocamente dal suo codice di ordine.
- **Storico Stato Ordine:** Ci permette di tenere traccia dello storico degli ordini. Identificato univocamente dall'ordine e dallo stato ad associato ad esso (mai ripetuto per lo stesso ordine).
- **Riga Ordine:** Singola voce all'interno di un ordine, che isola un prodotto specifico, la sua quantità e le esatte caratteristiche scelte dal cliente per quella specifica istanza. Identificato univocamente da un numero di riga (l'utente potrebbe aggiungere all'ordine due volte lo stesso prodotto, con la stessa quantità e le stesse modifiche).
- **Immagine:** Entità inserita per permettere ad un prodotto di avere più immagini. Un'immagine potrebbe essere indicata come quella da usare in copertina (**is_copertina**). Identificato univocamente dal percorso del file e dal prodotto a cui è associata.

2.2 Formalizzazione dei vincoli non esprimibili nel modello ER

1. **Progressione obbligata dello stato dell'ordine.** Il personale può cambiare lo stato di un ordine solo seguendo l'ordine *inserito* -> *in preparazione* -> *pronto* -> *in consegna* -> *consegnato*, senza possibilità di tornare a uno stato precedente e senza poter saltare stati intermedi. Ogni nuovo inserimento in Storico Stato Ordine deve quindi riferirsi allo stato immediatamente successivo a quello più recente registrato per lo stesso ordine.
2. **Coerenza temporale dello storico stati.** Il **timestamp_modifica** di ogni nuova riga di Storico Stato Ordine relativa a un dato ordine deve essere maggiore di tutti i **timestamp_modifica** già registrati per lo stesso ordine, e successivo a **data_inserimento** dell'ordine stesso.
3. **Unicità della caratteristica di default per gruppo.** All'interno di uno stesso Gruppo Caratteristica, al più una caratteristica può avere **is_default = true** e questa deve essere selezionata implicitamente se non viene selezionata un'altra caratteristica.

4. **Mutua esclusione in fase di selezione ordine.** In una stessa Riga Ordine, tra le caratteristiche selezionate tramite Personalizzata non possono comparirne due appartenenti allo stesso Gruppo Caratteristica.
 5. **Coerenza tra prezzo congelato e prezzo di listino al momento dell'ordine.** `prezzo_base_al_momento` (su Riga Ordine) e `diff_prezzo_al_momento` (su Personalizzata) devono corrispondere ai valori di `prezzo_base` e `differenza_prezzo` effettivamente in vigore su Prodotto/Caratteristica nell'istante di creazione dell'ordine.
 6. **Non negatività di importi e quantità.** `prezzo_base`, `differenza_prezzo`, `quantità`, `prezzo_base_al_momento` non possono assumere valori negativi e `tempo_preparazione` deve essere maggiore di zero.
 7. **Orario di consegna richiesto coerente.** Se specificato dal Cliente, l'`orario_richiesto` per un Ordine deve essere successivo a `data_inserimento` + `tempo_consegna_stimato`, la cucina deve avere il tempo di prepararlo e consegnarlo.
 8. **Unicità del ruolo per il Proprietario.** Il Proprietario deve essere unico nel sistema (pre-caricato all'installazione).
 9. **Congruenza del flag `is_copertina`.** Per ogni Prodotto che ha almeno un'immagine, tra le immagini associate solo una può avere `is_copertina = true`.
 10. **Selezione caratteristiche valide.** Un ordine può essere personalizzato solo con caratteristiche che sono effettivamente associate al prodotto di quella riga.
-

3. Progettazione logica

3.1 Ristrutturazione ed ottimizzazione del modello ER

Diagramma ER ristrutturato

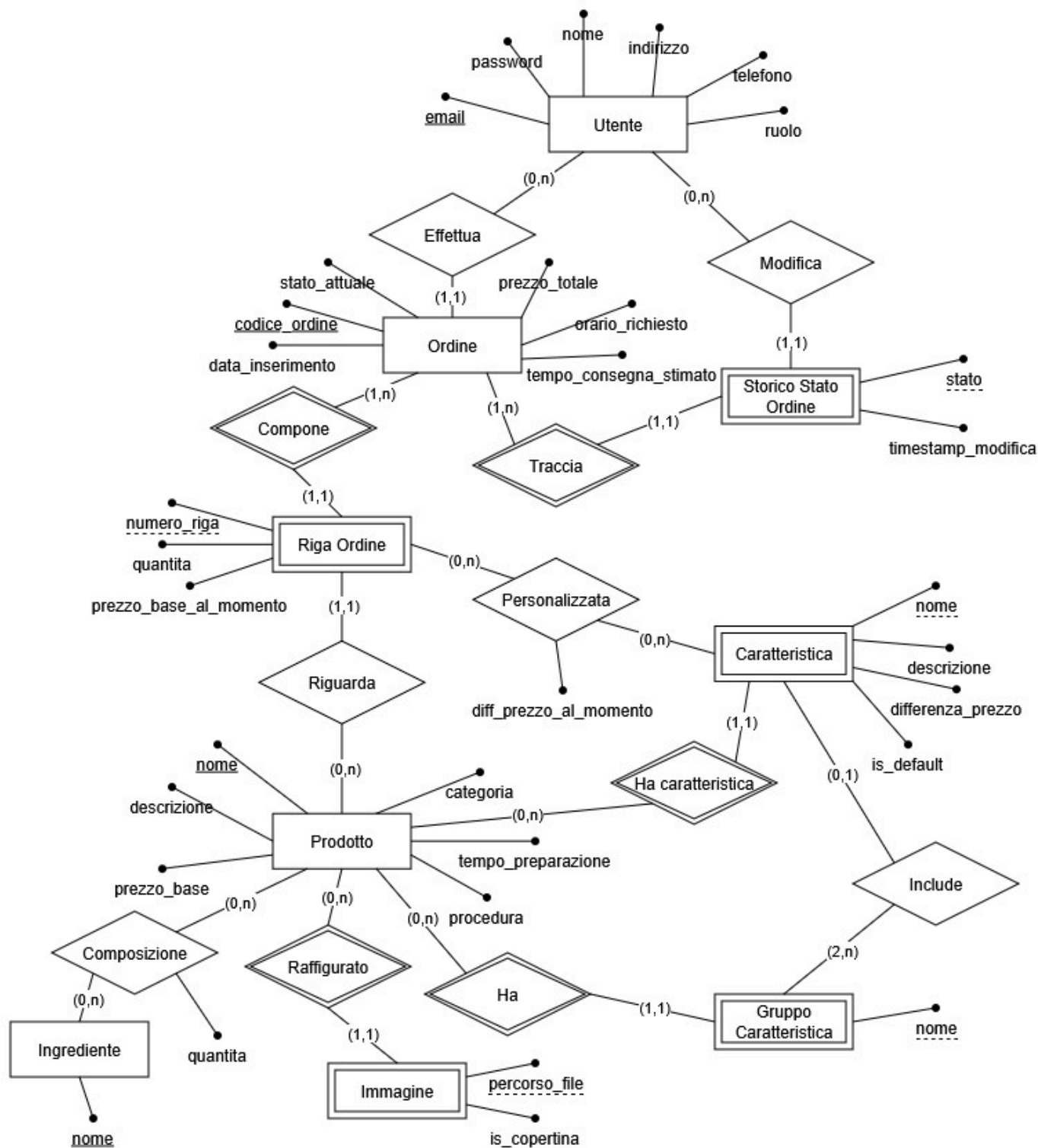


Diagramma realizzato con draw.io

Modifiche apportate

Eliminazione della generalizzazione Utente Il primo modello ER presentava una generalizzazione in cui Utente è entità padre di Cliente, Personale, Proprietario. Analizzando le sottocalissi, abbiamo notato che solo Cliente possiede degli attributi specifici (**telefono** e **indirizzo**) mentre Personale e Proprietario non ne hanno, entrambi ereditano direttamente gli attributi di Utente. Inoltre abbiamo riflettuto sulla logica di login, che deve essere uguale per ogni tipo di Utente indipendentemente dal ruolo, e se dividessimo le sottoclassi in tre entità distinte, il login diventerebbe molto complicato. Infatti invece di

effettuare un'unica query su un'unica tabella, si dovrebbero interrogare tre tabelle diverse o sapere già prima autenticazione a quale ruolo appartiene l'utente.

Abbiamo, quindi, scelto di risolvere questa gerarchia facendo una fusione dal basso verso l'alto (figli-genitore) accorpendo quindi le sottoclassi nella superclasse. Adesso abbiamo un'unica entità Utente a cui aggiungiamo un attributo discriminante **ruolo** (che potrà essere "cliente", "personale" o "proprietario"). I due attributi inizialmente esclusivi di Cliente **telefono** e **indirizzo**, verranno resi opzionali per Personale e Proprietario e obbligatori solo per Cliente. Un Ordine può essere collegato solo a un Utente con ruolo = 'cliente' e "una riga di Storico Stato Ordine può essere collegata solo a un Utente con ruolo = 'personale'".

Introduzione di ridondanze controllate Per ottimizzare il database e snellire le operazioni di lettura, abbiamo deciso di aggiungere delle ridondanze controllate a Ordine che avevamo rimosso nel modello concettuale:

- **stato_attuale**, sarebbe lo **stato** in Storico Stato Ordine più recente associato a un determinato Ordine. Utile per la visualizzazione degli ordini attivi per il personale. Viene aggiornato da un Trigger ogni volta che viene inserito un nuovo **stato** in Storico Stato Ordine, ovvero un membro del personale modifica lo stato di un ordine, piuttosto che ricavarlo ogni volta. Si dovrebbe fare ogni volta una JOIN con Storico Stato Ordine e cercare il MAX(timestamp_modifica) per ricavarlo.
- **prezzo_totale**, sarebbe la somma di tutti i prezzi base dei prodotti inclusi in un Ordine, moltiplicati per le quantità selezionate e con eventuale aggiunta della differenza di prezzo per la personalizzazione. invece di ricalcolarlo ogni volta, viene aggiornato da un trigger al momento della conferma di un ordine da parte del cliente. Utile anche per alleggerire il calcolo delle statistiche.
- **tempo_consegna_stimato**, sarebbe il tempo di preparazione più lungo di tutti i prodotti di un ordine, sommato ad un tempo di tragitto standard. Invece di ricavarlo ogni volta, conviene salvarlo una sola volta dato che è solo una stima visibile al cliente al momento della conferma di un ordine e non un dato che deve essere aggiornato. Quindi verrà calcolato e salvato tramite un Trigger all'inserimento dei prodotti nella Riga_Ordine o tramite una procedura all'atto della conferma dell'ordine.

3.2 Traduzione del modello ER nel modello relazionale

Le **chiavi primarie** sono indicate in grassetto e sottolineate, le **chiavi esterne** sono solo sottolineate. I vincoli di unicità derivati dalle ex-chiavi primarie concettuali sono indicati dalla dicitura **UNIQUE**.

- **Utente(id_utente, email, password, nome, ruolo, telefono, indirizzo)**
 - Vincoli: email UNIQUE
- **Ordine(id_ordine, codice_ordine, id_utente_cliente, data_inserimento, orario_richiesto, stato_attuale, prezzo_totale, tempo_consegna_stimato)**
 - Vincoli: codice_ordine UNIQUE
 - Chiavi esterne: id_utente_cliente (Utente)
- **Storico_Stato(id_storico, id_ordine, stato, id_utente_personale, timestamp_modifica)**
 - Vincoli: (id_ordine, stato) UNIQUE
 - Chiavi esterne: id_ordine (Ordine); id_utente_personale (Utente)
- **Prodotto(id_prodotto, nome, descrizione, prezzo_base, categoria, tempo_preparazione, procedura)**

- Vincoli: nome UNIQUE
- **Immagine**(id_immagine, id_prodotto, percorso_file, is_copertina)
 - Vincoli: (id_prodotto, percorso_file) UNIQUE
 - Chiavi esterne: id_prodotto (Prodotto)
- **Ingrediente**(id_ingrediente, nome)
 - Vincoli: nome UNIQUE
- **Composizione**(id_prodotto, id_ingrediente, quantita)
 - Chiavi esterne: id_prodotto (Prodotto); id_ingrediente (Ingrediente)
- **Gruppo_Caratteristica**(id_gruppo, id_prodotto, nome)
 - Vincoli: (id_prodotto, nome) UNIQUE
 - Chiavi esterne: id_prodotto (Prodotto)
- **Caratteristica**(id_caratteristica, id_prodotto, id_gruppo, nome, descrizione, differenza_prezzo, is_default)
 - Vincoli: (id_prodotto, nome) UNIQUE
 - Chiavi Esterne: id_prodotto (Prodotto); id_gruppo (Gruppo_Caratteristica)
- **Riga_Ordine**(id_riga, id_ordine, numero_riga, id_prodotto, quantita, prezzo_base_al_momento)
 - Vincoli: (id_ordine, numero_riga) UNIQUE
 - Chiavi Esterne: id_ordine (Ordine); id_prodotto (Prodotto)
- **Personalizzata**(id_riga, id_caratteristica, diff_prezzo_al_momento)
 - Chiavi Esterne: id_riga (Riga_Ordine); id_caratteristica (Caratteristica)

Tutte le chiavi esterne devono essere NOT NULL, ad eccezione di **id_gruppo** in Caratteristica che ammette anche NULL.

4. Progettazione fisica

4.1 Implementazione del modello relazionale

create.sql

```
CREATE DATABASE IF NOT EXISTS Delivery;
USE Delivery;

DROP TABLE IF EXISTS Personalizzata;
DROP TABLE IF EXISTS Riga_Ordine;
DROP TABLE IF EXISTS Storico_Stato;
DROP TABLE IF EXISTS Ordine;
DROP TABLE IF EXISTS Caratteristica;
DROP TABLE IF EXISTS Gruppo_Caratteristica;
DROP TABLE IF EXISTS Immagine;
DROP TABLE IF EXISTS Composizione;
DROP TABLE IF EXISTS Ingrediente;
DROP TABLE IF EXISTS Prodotto;
DROP TABLE IF EXISTS Utente;

CREATE TABLE Utente (
  id_utente      INT AUTO_INCREMENT PRIMARY KEY,
  email          VARCHAR(255) NOT NULL,
  password       VARCHAR(255) NOT NULL,
```

```

    nome            VARCHAR(100) NOT NULL,
    ruolo           ENUM('cliente', 'personale', 'proprietario') NOT NULL,
    telefono        VARCHAR(20) NULL,
    indirizzo       VARCHAR(255) NULL,
    CONSTRAINT uq_utente_email UNIQUE (email),
    -- telefono e indirizzo obbligatori solo per i clienti
    CONSTRAINT chk_utente_cliente_dati CHECK (
        ruolo <> 'cliente' OR (telefono IS NOT NULL AND indirizzo IS NOT
NULL)
    )
);

CREATE TABLE Prodotto (
    id_prodotto     INT AUTO_INCREMENT PRIMARY KEY,
    nome            VARCHAR(100) NOT NULL,
    descrizione     TEXT NOT NULL,
    prezzo_base     DECIMAL(6,2) NOT NULL,
    categoria       VARCHAR(50) NULL,
    tempo_preparazione INT NOT NULL COMMENT 'minuti',
    procedura       TEXT NULL,
    CONSTRAINT uq_prodotto_nome UNIQUE (nome),
    CONSTRAINT chk_prodotto_prezzo CHECK (prezzo_base >= 0),
    CONSTRAINT chk_prodotto_tempo CHECK (tempo_preparazione > 0)
);

CREATE TABLE Ingrediente (
    id_ingrediente  INT AUTO_INCREMENT PRIMARY KEY,
    nome            VARCHAR(100) NOT NULL,
    CONSTRAINT uq_ingrediente_nome UNIQUE (nome)
);

CREATE TABLE Composizione (
    id_prodotto     INT NOT NULL,
    id_ingrediente  INT NOT NULL,
    quantita        DECIMAL(6,2) NOT NULL,
    PRIMARY KEY (id_prodotto, id_ingrediente),
    CONSTRAINT fk_composizione_prodotto FOREIGN KEY (id_prodotto)
        REFERENCES Prodotto(id_prodotto) ON DELETE CASCADE ON UPDATE
CASCADE,
    CONSTRAINT fk_composizione_ingrediente FOREIGN KEY (id_ingrediente)
        REFERENCES Ingrediente(id_ingrediente) ON DELETE RESTRICT ON UPDATE
CASCADE,
    CONSTRAINT chk_composizione_quantita CHECK (quantita > 0)
);

CREATE TABLE Immagine (
    id_immagine     INT AUTO_INCREMENT PRIMARY KEY,
    id_prodotto     INT NOT NULL,
    percorso_file   VARCHAR(255) NOT NULL,
    is_copertina    BOOLEAN NOT NULL DEFAULT FALSE,
    CONSTRAINT uq_immagine_percorso UNIQUE (id_prodotto, percorso_file),
    CONSTRAINT fk_immagine_prodotto FOREIGN KEY (id_prodotto)
        REFERENCES Prodotto(id_prodotto) ON DELETE CASCADE ON UPDATE
CASCADE

```

```

);

CREATE TABLE Gruppo_Caratteristica (
    id_gruppo    INT AUTO_INCREMENT PRIMARY KEY,
    id_prodotto  INT NOT NULL,
    nome         VARCHAR(100) NOT NULL,
    CONSTRAINT uq_gruppo_prodotto_nome UNIQUE (id_prodotto, nome),
    CONSTRAINT fk_gruppo_prodotto FOREIGN KEY (id_prodotto)
        REFERENCES Prodotto(id_prodotto) ON DELETE CASCADE ON UPDATE
CASCADE
);

CREATE TABLE Caratteristica (
    id_caratteristica INT AUTO_INCREMENT PRIMARY KEY,
    id_prodotto       INT NOT NULL,
    id_gruppo         INT NULL,
    nome              VARCHAR(100) NOT NULL,
    descrizione       TEXT NULL,
    differenza_prezzo  DECIMAL(6,2) NOT NULL DEFAULT 0,
    is_default        BOOLEAN NOT NULL DEFAULT FALSE,
    CONSTRAINT uq_caratteristica_prodotto_nome UNIQUE (id_prodotto, nome),
    CONSTRAINT fk_caratteristica_prodotto FOREIGN KEY (id_prodotto)
        REFERENCES Prodotto(id_prodotto) ON DELETE CASCADE ON UPDATE
CASCADE,
    CONSTRAINT fk_caratteristica_gruppo FOREIGN KEY (id_gruppo)
        REFERENCES Gruppo_Caratteristica(id_gruppo) ON DELETE SET NULL ON
UPDATE CASCADE
);

CREATE TABLE Ordine (
    id_ordine          INT AUTO_INCREMENT PRIMARY KEY,
    codice_ordine      VARCHAR(30) NOT NULL,
    id_utente_cliente  INT NOT NULL,
    data_inserimento   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    orario_richiesto   DATETIME NULL,
    stato_attuale      ENUM('inserito','in preparazione','pronto','in
consegna','consegnato')
        NOT NULL DEFAULT 'inserito',
    prezzo_totale      DECIMAL(8,2) NULL,
    tempo_consegna_stimato INT NULL COMMENT 'minuti',
    CONSTRAINT uq_ordine_codice UNIQUE (codice_ordine),
    CONSTRAINT fk_ordine_cliente FOREIGN KEY (id_utente_cliente)
        REFERENCES Utente(id_utente) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT chk_ordine_orario CHECK (orario_richiesto IS NULL OR
orario_richiesto > data_inserimento)
);

CREATE TABLE Storico_Stato (
    id_storico        INT AUTO_INCREMENT PRIMARY KEY,
    id_ordine          INT NOT NULL,
    stato             ENUM('inserito','in preparazione','pronto','in
consegna','consegnato') NOT NULL,
    id_utente_personale INT NOT NULL,
    timestamp_modifica DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

```

```

CONSTRAINT uq_storico_ordine_stato UNIQUE (id_ordine, stato),
CONSTRAINT fk_storico_ordine FOREIGN KEY (id_ordine)
    REFERENCES Ordine(id_ordine) ON DELETE CASCADE ON UPDATE CASCADE,
CONSTRAINT fk_storico_personale FOREIGN KEY (id_utente_personale)
    REFERENCES Utente(id_utente) ON DELETE RESTRICT ON UPDATE CASCADE
);

CREATE TABLE Riga_Ordine (
    id_riga                INT AUTO_INCREMENT PRIMARY KEY,
    id_ordine              INT NOT NULL,
    numero_riga            INT NOT NULL,
    id_prodotto            INT NOT NULL,
    quantita               INT NOT NULL,
    prezzo_base_al_momento DECIMAL(6,2) NOT NULL,
    CONSTRAINT uq_riga_ordine_numero UNIQUE (id_ordine, numero_riga),
    CONSTRAINT fk_riga_ordine FOREIGN KEY (id_ordine)
        REFERENCES Ordine(id_ordine) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_riga_prodotto FOREIGN KEY (id_prodotto)
        REFERENCES Prodotto(id_prodotto) ON DELETE RESTRICT ON UPDATE
CASCADE,
    CONSTRAINT chk_riga_quantita CHECK (quantita > 0),
    CONSTRAINT chk_riga_prezzo CHECK (prezzo_base_al_momento >= 0)
);

CREATE TABLE Personalizzata (
    id_riga                INT NOT NULL,
    id_caratteristica      INT NOT NULL,
    diff_prezzo_al_momento DECIMAL(6,2) NOT NULL DEFAULT 0,
    PRIMARY KEY (id_riga, id_caratteristica),
    CONSTRAINT fk_personalizzata_riga FOREIGN KEY (id_riga)
        REFERENCES Riga_Ordine(id_riga) ON DELETE CASCADE ON UPDATE
CASCADE,
    CONSTRAINT fk_personalizzata_caratteristica FOREIGN KEY
(id_caratteristica)
        REFERENCES Caratteristica(id_caratteristica) ON DELETE RESTRICT ON
UPDATE CASCADE
);

```

insert.sql

```

USE Delivery;

INSERT INTO Utente (id_utente, email, password, nome, ruolo, telefono,
indirizzo) VALUES
(1, 'admin@email.it', 'password_1', 'Amministratore', 'proprietario', NULL,
NULL),
(2, 'luca@personale.it', 'password_2', 'Luca B.', 'personale', NULL, NULL),
(3, 'sara@personale.it', 'password_3', 'Sara M.', 'personale', NULL, NULL),
(4, 'marco@personale.it', 'password_4', 'Marco T.', 'personale', NULL,
NULL),

```

```
(5, 'mario.rossi@email.it', 'password_5', 'Mario Rossi', 'cliente',
'3331234567', 'Via Roma 1, L''Aquila'),
(6, 'giulia.bianchi@email.it', 'password_6', 'Giulia Bianchi', 'cliente',
'3332222222', 'Via Milano 2, L''Aquila'),
(7, 'andrea.verdi@email.it', 'password_7', 'Andrea Verdi', 'cliente',
'3333333333', 'Via Torino 3, L''Aquila'),
(8, 'elena.conti@email.it', 'password_8', 'Elena Conti', 'cliente',
'3334444444', 'Via Napoli 4, L''Aquila');
```

```
INSERT INTO Prodotto (id_prodotto, nome, descrizione, prezzo_base,
categoria, tempo_preparazione, procedura) VALUES
(1, 'Bruschette miste', 'Quattro semplici e gustose bruschette al pomodoro,
con un goccio di olio EVO e foglie di basilico fresco.', 5.50, 'Antipasti',
8, 'Tostare il pane, strofinare con aglio, condire con pomodoro a cubetti,
olio e basilico.'),
(2, 'Tagliatelle al ragù', 'Pasta fresca all''uovo con ragù di carne cotto
lentamente per 4 ore.', 9.00, 'Primi', 15, 'Cuocere la pasta 3 minuti in
acqua salata, mantecare con il ragù caldo, spolverare di parmigiano.'),
(3, 'Pizza Margherita', 'La tipica pizza napoletana cotta nel forno a
legna.', 7.00, 'Pizze', 12, 'Stendere l''impasto, condire con pomodoro e
mozzarella, cuocere nel forno a legna a 400°C per 90 secondi, guarnire con
basilico.'),
(4, 'Pizza Diavola', 'L''iconica pizza piccante dal sapore deciso.', 8.00,
'Pizze', 12, 'Stendere l''impasto, condire con pomodoro, mozzarella e
salame, cuocere nel forno a legna, rifinire con olio piccante in uscita.'),
(5, 'Tiramisù della casa', 'Dolce con savoiardi, mascarpone e caffè fatto
da noi.', 4.50, 'Dolci', 3, 'Porzionare dalla vaschetta preparata in
mattinata, spolverare di cacao al momento del servizio.'),
(6, 'Caffè', 'Espresso della nostra torrefazione di fiducia.', 1.00,
'Bevande', 2, 'Estrazione singola da macinato fresco, 25 secondi circa.');
```

```
INSERT INTO Immagine (id_immagine, id_prodotto, percorso_file,
is_copertina) VALUES
(1, 1, 'img/bruschette.jpg', TRUE),
(2, 2, 'img/tagliatelle.jpg', TRUE),
(3, 3, 'img/pizza-margherita.jpg', TRUE),
(4, 3, 'img/pizza-margherita2.jpg', FALSE),
(5, 4, 'img/pizza-diavola.jpg', TRUE),
(6, 5, 'img/tiramisu.jpg', TRUE),
(7, 6, 'img/caffe.jpg', TRUE);
```

```
INSERT INTO Ingrediente (id_ingrediente, nome) VALUES
(1, 'Pane casereccio'), (2, 'Pomodoro'), (3, 'Basilico'), (4, 'Olio
extravergine'),
(5, 'Tagliatelle fresche'), (6, 'Ragù di carne'), (7, 'Parmigiano'), (8,
'Impasto pizza'),
(9, 'Pomodoro San Marzano'), (10, 'Fiordilatte'), (11, 'Salame piccante'),
(12, 'Olio al peperoncino'),
(13, 'Savoiardi'), (14, 'Crema al mascarpone'), (15, 'Caffè espresso'),
(16, 'Cacao amaro'), (17, 'Miscela espresso');
```

```
INSERT INTO Composizione (id_prodotto, id_ingrediente, quantita) VALUES
(1, 1, 200), (1, 2, 200), (1, 3, 5), (1, 4, 20),
(2, 5, 120), (2, 6, 150), (2, 7, 20),
```



```
(3, 8, 250), (3, 9, 80), (3, 10, 100), (3, 3, 5),  
(4, 8, 250), (4, 2, 80), (4, 10, 100), (4, 11, 60), (4, 12, 5),  
(5, 13, 100), (5, 14, 150), (5, 15, 30), (5, 16, 5),  
(6, 17, 7);
```

```
INSERT INTO Gruppo_Caratteristica (id_gruppo, id_prodotto, nome) VALUES  
(1, 2, 'Porzione'),  
(2, 3, 'Impasto'),  
(3, 4, 'Piccantezza'),  
(4, 6, 'Zucchero');
```

```
INSERT INTO Caratteristica (id_caratteristica, id_prodotto, id_gruppo,  
nome, descrizione, differenza_prezzo, is_default) VALUES  
(1, 2, 1, 'Normale', NULL, 0.00, TRUE),  
(2, 2, 1, 'Abbondante', NULL, 2.00, FALSE),  
(3, 3, 2, 'Classico', NULL, 0.00, TRUE),  
(4, 3, 2, 'Integrale', NULL, 0.50, FALSE),  
(5, 3, NULL, 'Extra mozzarella', NULL, 1.50, FALSE),  
(6, 3, NULL, 'Origano fresco', NULL, 0.30, FALSE),  
(7, 4, 3, 'Normale', NULL, 0.00, TRUE),  
(8, 4, 3, 'Extra piccante', NULL, 0.00, FALSE),  
(9, 6, 4, 'Senza zucchero', NULL, -0.05, FALSE),  
(10, 6, 4, 'Zuccherato', NULL, 0.00, TRUE),  
(11, 6, 4, 'Molto zuccherato', NULL, 0.00, FALSE),  
(12, 6, NULL, 'Con panna', NULL, 0.50, FALSE),  
(13, 6, NULL, 'Freddo', NULL, 1.00, FALSE);
```

```
INSERT INTO Ordine (id_ordine, codice_ordine, id_utente_cliente,  
data_inserimento, orario_richiesto) VALUES  
(1, 'ORD-2026-0001', 5, '2026-07-14 19:40:00', '2026-07-14 20:15:00'),  
(2, 'ORD-2026-0002', 6, '2026-07-14 19:20:00', '2026-07-14 20:00:00'),  
(3, 'ORD-2026-0003', 7, '2026-07-14 19:05:00', '2026-07-14 19:45:00'),  
(4, 'ORD-2026-0004', 8, '2026-07-14 18:50:00', '2026-07-14 19:30:00');
```

```
INSERT INTO Riga_Ordine (id_riga, id_ordine, numero_riga, id_prodotto,  
quantita, prezzo_base_al_momento) VALUES  
(1, 1, 1, 3, 1, 7.00), (2, 1, 2, 4, 1, 8.00), (3, 1, 3, 5, 2, 4.50),  
(4, 2, 1, 2, 1, 9.00), (5, 2, 2, 1, 1, 5.50),  
(6, 3, 1, 3, 2, 7.00), (7, 3, 2, 6, 2, 1.00),  
(8, 4, 1, 4, 1, 8.00), (9, 4, 2, 5, 1, 4.50);
```

```
INSERT INTO Personalizzata (id_riga, id_caratteristica,  
diff_prezzo_al_momento) VALUES  
(1, 4, 0.50),  
(2, 8, 0.00),  
(4, 2, 2.00),  
(7, 10, 0.00);
```

```
INSERT INTO Storico_Stato (id_storico, id_ordine, stato,  
id_utente_personale, timestamp_modifica) VALUES  
(1, 1, 'in preparazione', 2, '2026-07-14 19:52:00'),  
(2, 2, 'in preparazione', 2, '2026-07-14 19:30:00'),  
(3, 3, 'in preparazione', 3, '2026-07-14 19:10:00'),  
(4, 3, 'pronto', 3, '2026-07-14 19:24:00'),
```

```
(5, 4, 'in preparazione', 2, '2026-07-14 18:55:00'),  
(6, 4, 'pronto', 2, '2026-07-14 19:08:00'),  
(7, 4, 'in consegna', 4, '2026-07-14 19:12:00');
```

4.2 Implementazione dei vincoli

Controllo avanzamento stato e storicizzazione ordini

```
USE Delivery;  
  
DELIMITER //  
  
-- Trigger per il controllo dell'avanzamento dello stato e del timestamp  
DROP TRIGGER IF EXISTS chk_avanzamento_stato_ordine //  
CREATE TRIGGER chk_avanzamento_stato_ordine  
BEFORE INSERT ON Storico_Stato  
FOR EACH ROW  
BEGIN  
    DECLARE ultimo_stato_enum INT;  
    DECLARE order_date DATETIME;  
  
    SELECT CAST(stato_attuale AS UNSIGNED), data_inserimento  
    INTO ultimo_stato_enum, order_date  
    FROM Ordine WHERE id_ordine = NEW.id_ordine;  
  
    -- Controllo temporale (vincolo 2)  
    IF NEW.timestamp_modifica <= order_date THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Errore: Il timestamp non può essere precedente  
alla data di creazione dell''ordine.';  
    END IF;  
  
    -- Controllo progressione (vincolo 1)  
    IF CAST(NEW.stato AS UNSIGNED) != (ultimo_stato_enum + 1) THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Errore: Transizione di stato non valida o salto  
di stato non permesso.';  
    END IF;  
END //  
  
-- Trigger per l'aggiornamento della ridondanza stato_attuale su Ordine  
DROP TRIGGER IF EXISTS update_ridondanza_stato //  
CREATE TRIGGER update_ridondanza_stato  
AFTER INSERT ON Storico_Stato  
FOR EACH ROW  
BEGIN  
    UPDATE Ordine SET stato_attuale = NEW.stato WHERE id_ordine =  
NEW.id_ordine;  
END //
```

```
DELIMITER ;
```

Qui viene anche aggiornata in automatico la ridondanza `stato_attuale`.

Vincoli di ruolo sugli utenti

```
DELIMITER //

-- Un Ordine deve essere collegato a un Utente con ruolo = 'cliente'
DROP TRIGGER IF EXISTS chk_ruolo_cliente_ordine //
CREATE TRIGGER chk_ruolo_cliente_ordine
BEFORE INSERT ON Ordine
FOR EACH ROW
BEGIN
    DECLARE ruolo_utente VARCHAR(20);

    SELECT ruolo INTO ruolo_utente
    FROM Utente WHERE id_utente = NEW.id_utente_cliente;

    IF ruolo_utente <> 'cliente' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Errore: un Ordine può essere collegato solo a
un Utente con ruolo cliente.';
    END IF;
END //

-- Una riga di Storico_Stato deve essere collegata a un Utente con ruolo =
'personale'
-- Eseguito prima di chk_avanzamento_stato_ordine, per dare priorità
-- a un eventuale errore sul ruolo rispetto a uno sulla progressione.
DROP TRIGGER IF EXISTS chk_ruolo_personale_storico //
CREATE TRIGGER chk_ruolo_personale_storico
BEFORE INSERT ON Storico_Stato
FOR EACH ROW
PRECEDES chk_avanzamento_stato_ordine
BEGIN
    DECLARE ruolo_utente VARCHAR(20);

    SELECT ruolo INTO ruolo_utente
    FROM Utente WHERE id_utente = NEW.id_utente_personale;

    IF ruolo_utente <> 'personale' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Errore: una riga di Storico_Stato può essere
collegata solo a un Utente con ruolo personale.';
    END IF;
END //

-- Unicità del Proprietario (vincolo 8)
```

```
DROP TRIGGER IF EXISTS chk_proprietario_unico //
CREATE TRIGGER chk_proprietario_unico
BEFORE INSERT ON Utente
FOR EACH ROW
BEGIN
    DECLARE num_proprietari INT;
    IF NEW.ruolo = 'proprietario' THEN
        SELECT COUNT(*) INTO num_proprietari FROM Utente WHERE ruolo =
'proprietario';
        IF num_proprietari > 0 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Errore: Esiste già un proprietario
registrato nel sistema.';
        END IF;
    END IF;
END //

DELIMITER ;
```

Gestione Riga Ordine, congelamento prezzi e tempi stimati

```
DELIMITER //

-- Trigger su Riga_Ordine
DROP TRIGGER IF EXISTS before_insert_riga_ordine //
CREATE TRIGGER before_insert_riga_ordine
BEFORE INSERT ON Riga_Ordine
FOR EACH ROW
BEGIN
    DECLARE v_prezzo DECIMAL(6,2);
    DECLARE v_tempo_prep INT;

    SELECT prezzo_base, tempo_preparazione
    INTO v_prezzo, v_tempo_prep
    FROM Prodotto WHERE id_prodotto = NEW.id_prodotto;

    -- Congela il prezzo base (vincolo 5)
    SET NEW.prezzo_base_al_momento = v_prezzo;

    -- Aggiorna il tempo stimato sull'ordine (con 15 min di tragitto)
    UPDATE Ordine
    SET tempo_consegna_stimato = GREATEST(IFNULL(tempo_consegna_stimato,
0), v_tempo_prep + 15)
    WHERE id_ordine = NEW.id_ordine;
END //

-- Trigger per aggiornare il prezzo totale dell'ordine all'aggiunta di una
riga
DROP TRIGGER IF EXISTS update_prezzo_totale_riga //
CREATE TRIGGER update_prezzo_totale_riga
```

```
AFTER INSERT ON Riga_Ordine
FOR EACH ROW
BEGIN
    UPDATE Ordine
    SET prezzo_totale = IFNULL(prezzo_totale, 0) +
    (NEW.prezzo_base_al_momento * NEW.quantita)
    WHERE id_ordine = NEW.id_ordine;
END //

-- Trigger su Personalizzata
DROP TRIGGER IF EXISTS chk_inserimento_personalizzata_completo //
CREATE TRIGGER chk_inserimento_personalizzata_completo
BEFORE INSERT ON Personalizzata
FOR EACH ROW
BEGIN
    DECLARE prodotto_riga INT;
    DECLARE prodotto_car INT;
    DECLARE gruppo_car INT;
    DECLARE overlap_gruppo INT;
    DECLARE prezzo_diff DECIMAL(6,2);

    SELECT id_prodotto INTO prodotto_riga FROM Riga_Ordine WHERE id_riga =
    NEW.id_riga;
    SELECT id_prodotto, id_gruppo, differenza_prezzo INTO prodotto_car,
    gruppo_car, prezzo_diff
    FROM Caratteristica WHERE id_caratteristica = NEW.id_caratteristica;

    -- Congela la differenza di prezzo (vincolo 5)
    SET NEW.diff_prezzo_al_momento = prezzo_diff;

    -- Controllo coerenza prodotto (vincolo 10)
    IF prodotto_riga != prodotto_car THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Errore: La caratteristica non appartiene al
    prodotto della riga.';
    END IF;

    -- Controllo mutua esclusione (vincolo 4)
    IF gruppo_car IS NOT NULL THEN
        SELECT COUNT(*) INTO overlap_gruppo
        FROM Personalizzata p
        JOIN Caratteristica c ON p.id_caratteristica = c.id_caratteristica
        WHERE p.id_riga = NEW.id_riga AND c.id_gruppo = gruppo_car;

        IF overlap_gruppo > 0 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Errore: Mutua esclusione violata per questo
    gruppo.';
        END IF;
    END IF;
END //

-- Trigger per aggiornare il prezzo totale quando si aggiunge una
personalizzazione
```

```

DROP TRIGGER IF EXISTS update_prezzo_totale_personalizzata //
CREATE TRIGGER update_prezzo_totale_personalizzata
AFTER INSERT ON Personalizzata
FOR EACH ROW
BEGIN
    DECLARE id_ord INT;
    DECLARE qta INT;

    SELECT id_ordine, quantita INTO id_ord, qta
    FROM Riga_Ordine WHERE id_riga = NEW.id_riga;

    UPDATE Ordine
    SET prezzo_totale = IFNULL(prezzo_totale, 0) +
    (NEW.diff_prezzo_al_momento * qta)
    WHERE id_ordine = id_ord;
END //

DELIMITER ;

```

Qui vengono anche aggiornate in automatico le ridondanze `prezzo_totale` e `tempo_consegna_stimato`.

Coerenza tra Caratteristica e Gruppo Caratteristica

```

DELIMITER //

-- Coerenza tra il gruppo di una caratteristica e il prodotto a cui
appartiene
DROP TRIGGER IF EXISTS chk_coerenza_gruppo_prodotto_insert //
CREATE TRIGGER chk_coerenza_gruppo_prodotto_insert
BEFORE INSERT ON Caratteristica
FOR EACH ROW
BEGIN
    DECLARE prodotto_gruppo INT;

    IF NEW.id_gruppo IS NOT NULL THEN
        SELECT id_prodotto INTO prodotto_gruppo
        FROM Gruppo_Caratteristica WHERE id_gruppo = NEW.id_gruppo;

        IF prodotto_gruppo <> NEW.id_prodotto THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Errore: il gruppo indicato appartiene a un
prodotto diverso da quello della caratteristica.';
        END IF;
    END IF;
END //

DROP TRIGGER IF EXISTS chk_coerenza_gruppo_prodotto_update //
CREATE TRIGGER chk_coerenza_gruppo_prodotto_update
BEFORE UPDATE ON Caratteristica

```



```
FOR EACH ROW
BEGIN
    DECLARE prodotto_gruppo INT;

    IF NEW.id_gruppo IS NOT NULL THEN
        SELECT id_prodotto INTO prodotto_gruppo
        FROM Gruppo_Caratteristica WHERE id_gruppo = NEW.id_gruppo;

        IF prodotto_gruppo <> NEW.id_prodotto THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Errore: il gruppo indicato appartiene a un
prodotto diverso da quello della caratteristica.';
        END IF;
    END IF;
END //

DELIMITER ;
```

Esclusività e coerenza sul menu

```
DELIMITER //

-- Esclusività Immagine di Copertina – INSERT (vincolo 9)
DROP TRIGGER IF EXISTS chk_unica_copertina_insert //
CREATE TRIGGER chk_unica_copertina_insert
BEFORE INSERT ON Immagine
FOR EACH ROW
BEGIN
    IF NEW.is_copertina = TRUE THEN
        UPDATE Immagine SET is_copertina = FALSE WHERE id_prodotto =
NEW.id_prodotto;
    END IF;
END //

-- Esclusività Immagine di Copertina – UPDATE
DROP TRIGGER IF EXISTS chk_unica_copertina_update //
CREATE TRIGGER chk_unica_copertina_update
BEFORE UPDATE ON Immagine
FOR EACH ROW
BEGIN
    IF NEW.is_copertina = TRUE THEN
        UPDATE Immagine
        SET is_copertina = FALSE
        WHERE id_prodotto = NEW.id_prodotto
        AND id_immagine <> NEW.id_immagine;
    END IF;
END //

-- Esclusività Caratteristica di Default – INSERT (vincolo 3)
DROP TRIGGER IF EXISTS chk_unico_default_insert //
```

```

CREATE TRIGGER chk_unico_default_insert
BEFORE INSERT ON Caratteristica
FOR EACH ROW
BEGIN
    IF NEW.is_default = TRUE AND NEW.id_gruppo IS NOT NULL THEN
        UPDATE Caratteristica SET is_default = FALSE WHERE id_gruppo =
NEW.id_gruppo;
    END IF;
END //

-- Esclusività Caratteristica di Default - UPDATE
DROP TRIGGER IF EXISTS chk_unico_default_update //
CREATE TRIGGER chk_unico_default_update
BEFORE UPDATE ON Caratteristica
FOR EACH ROW
BEGIN
    IF NEW.is_default = TRUE AND NEW.id_gruppo IS NOT NULL THEN
        UPDATE Caratteristica
        SET is_default = FALSE
        WHERE id_gruppo = NEW.id_gruppo
        AND id_caratteristica <> NEW.id_caratteristica;
    END IF;
END //

DELIMITER ;

```

Coerenza dell'orario richiesto

```

DELIMITER //

-- Coerenza Orario Richiesto (vincolo 7)
-- orario_richiesto viene sempre impostato in un secondo momento
-- rispetto alla creazione dell'ordine (alla conferma), quindi il controllo
copre solo BEFORE UPDATE.

DROP TRIGGER IF EXISTS chk_orario_richiesto //
CREATE TRIGGER chk_orario_richiesto
BEFORE UPDATE ON Ordine
FOR EACH ROW
BEGIN
    IF NEW.orario_richiesto IS NOT NULL AND NEW.orario_richiesto !=
IFNULL(OLD.orario_richiesto, '1000-01-01') THEN
        IF NEW.orario_richiesto <= DATE_ADD(NEW.data_inserimento, INTERVAL
NEW.tempo_consegna_stimato MINUTE) THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Errore: Orario richiesto incompatibile con
i tempi di preparazione.';
        END IF;
    END IF;
END //

```

```
DELIMITER ;
```

4.3 Implementazione funzionalità richieste

Funzionalità 1

Generazione del menu (lista dei prodotti con tutte le informazioni visibili al cliente, possibilmente anche le relative caratteristiche con la differenza di prezzo).

```
SELECT
    p.categoria,
    p.nome AS nome_prodotto,
    p.descrizione AS descrizione_prodotto,
    p.prezzo_base,
    i.percorso_file AS immagine_copertina,
    gc.nome AS gruppo_mutua_esclusione,
    c.nome AS nome_caratteristica,
    c.descrizione AS descrizione_caratteristica,
    c.differenza_prezzo,
    c.is_default
FROM
    Prodotto p
LEFT JOIN
    Immagine i ON p.id_prodotto = i.id_prodotto AND i.is_copertina = TRUE
LEFT JOIN
    Caratteristica c ON p.id_prodotto = c.id_prodotto
LEFT JOIN
    Gruppo_Caratteristica gc ON c.id_gruppo = gc.id_gruppo
ORDER BY
    p.categoria,
    p.nome,
    gc.nome,
    c.is_default DESC,
    c.nome;
```

Funzionalità 2

Eliminazione di una caratteristica associata a un prodotto.

```
DELETE FROM Caratteristica
WHERE
    nome = 'Senza Zucchero'
    AND id_prodotto = (SELECT id_prodotto FROM Prodotto WHERE nome =
    'Caffè');
```

Funzionalità 3

Inserimento di un prodotto in un ordine, comprensivo delle sue eventuali caratteristiche.

```
START TRANSACTION;

SET @next_riga = (SELECT COALESCE(MAX(numero_riga), 0) + 1 FROM Riga_Ordine
WHERE id_ordine = 5);

INSERT INTO Riga_Ordine (id_ordine, numero_riga, id_prodotto, quantita)
VALUES (5, @next_riga, 10, 2);

SET @id_nuova_riga = LAST_INSERT_ID();

INSERT INTO Personalizzata (id_riga, id_caratteristica)
VALUES (@id_nuova_riga, 12);

INSERT INTO Personalizzata (id_riga, id_caratteristica)
VALUES (@id_nuova_riga, 15);

COMMIT;
```

Funzionalità 4

Calcolo del tempo stimato di consegna e del prezzo totale di un ordine (*suggerimento: potete provare a usare una sotto-query per calcolare la differenza cumulativa di prezzo derivante dalle caratteristiche selezionate e poi sommarla al prezzo base*).

```
SELECT
    prezzo_totale,
    tempo_consegna_stimato
FROM
    Ordine
WHERE
    id_ordine = 5;
```

Funzionalità 5

Lista degli ordini non ancora messi in preparazione dopo più di un'ora dall'inserimento (*suggerimento: è quindi necessario prevedere degli opportuni timestamp da affiancare agli stati*).

```
SELECT
    id_ordine,
    codice_ordine,
    data_inserimento,
    id_utente_cliente
FROM
```

```
Ordine
WHERE
  stato_attuale = 'inserito'
  AND data_inserimento <= DATE_SUB(NOW(), INTERVAL 1 HOUR)
ORDER BY
  data_inserimento ASC;
```

Funzionalità 6

Calcolo del tempo medio di consegna, cioè di passaggio tra lo stato *in consegna* in quello *consegnato*, per ciascun membro del personale addetto alla consegna (*supponiamo che chi consegna sia colui il quale imposta lo stato su consegnato*).

```
SELECT
  u.nome AS fattorino,
  u.email,
  ROUND(AVG(TIMESTAMPDIFF(MINUTE, s_inizio.timestamp_modifica,
s_fine.timestamp_modifica)), 1) AS tempo_medio_consegna_minuti,
  COUNT(s_fine.id_ordine) AS numero_consegne_effettuate
FROM
  Storico_Stato s_fine
JOIN
  Storico_Stato s_inizio ON s_fine.id_ordine = s_inizio.id_ordine
JOIN
  Utente u ON s_fine.id_utente_personale = u.id_utente
WHERE
  s_fine.stato = 'consegnato'
  AND s_inizio.stato = 'in consegna'
GROUP BY
  u.id_utente, u.nome, u.email
ORDER BY
  tempo_medio_consegna_minuti ASC;
```

Funzionalità 7

Classifica di gradimento dei prodotti (*quali prodotti compaiono più comunemente negli ordini?*).

```
SELECT
  p.nome AS nome_prodotto,
  p.categoria,
  SUM(ro.quantita) AS quantita_totale_venduta,
  COUNT(DISTINCT ro.id_ordine) AS numero_ordini_distinti
FROM
  Riga_Ordine ro
JOIN
  Prodotto p ON ro.id_prodotto = p.id_prodotto
GROUP BY
  p.id_prodotto, p.nome, p.categoria
```

```
ORDER BY
    quantita_totale_venduta DESC;
```

Funzionalità 8

Calcolo dell'incasso giornaliero.

```
SELECT
    DATE(ss.timestamp_modifica) AS data_riferimento,
    SUM(o.prezzo_totale) AS incasso_giornaliero,
    COUNT(o.id_ordine) AS numero_ordini_evasi
FROM
    Ordine o
JOIN
    Storico_Stato ss ON o.id_ordine = ss.id_ordine
WHERE
    ss.stato = 'consegnato'
    AND DATE(ss.timestamp_modifica) = CURRENT_DATE()
GROUP BY
    DATE(ss.timestamp_modifica);
```

Funzionalità 9

Prospetto del consumo di ingredienti in un anno (*quantità di ciascun ingrediente consumata in un certo anno*).

```
SELECT
    i.nome AS nome_ingrediente,
    SUM(ro.quantita * c.quantita) AS quantita_totale_consumata
FROM
    Ordine o
JOIN
    Storico_Stato ss ON o.id_ordine = ss.id_ordine AND ss.stato =
    'consegnato'
JOIN
    Riga_Ordine ro ON o.id_ordine = ro.id_ordine
JOIN
    Composizione c ON ro.id_prodotto = c.id_prodotto
JOIN
    Ingrediente i ON c.id_ingrediente = i.id_ingrediente
WHERE
    YEAR(ss.timestamp_modifica) = 2026
GROUP BY
    i.id_ingrediente, i.nome
ORDER BY
    quantita_totale_consumata DESC;
```

Funzionalità 10

Estrazione dei prodotti preferiti da un cliente (cioè i prodotti più ordinati da quel cliente, magari escludendo a priori quelli ordinati solo un paio di volte...).

```
SET @id_cliente = 2; -- parametro: cliente da analizzare

SELECT
    p.nome AS prodotto_preferito,
    SUM(ro.quantita) AS totale_pezzi_acquistati,
    COUNT(ro.id_ordine) AS numero_volte_ordinato
FROM
    Ordine o
JOIN
    Riga_Ordine ro ON o.id_ordine = ro.id_ordine
JOIN
    Prodotto p ON ro.id_prodotto = p.id_prodotto
WHERE
    o.id_utente_cliente = @id_cliente
    AND o.stato_attuale = 'consegnato'
GROUP BY
    p.id_prodotto, p.nome
HAVING
    COUNT(ro.id_ordine) >= 3
ORDER BY
    numero_volte_ordinato DESC,
    totale_pezzi_acquistati DESC;
```

Funzionalità 11

Conteggio degli ordini attivi (non in stato *consegnato*) divisi per il loro stato di avanzamento.

```
SELECT
    stato_attuale AS stato_ordine,
    COUNT(id_ordine) AS numero_ordini
FROM
    Ordine
WHERE
    stato_attuale != 'consegnato'
GROUP BY
    stato_attuale
ORDER BY
    CAST(stato_attuale AS UNSIGNED) ASC;
```

Funzionalità 12

Conteggio degli ordini smaltiti (consegnati) in uno specifico giorno.


```
SET @giorno_riferimento = '2026-07-14'; -- parametro: giorno da verificare

SELECT
    COUNT(o.id_ordine) AS ordini_smaltilti
FROM
    Ordine o
JOIN
    Storico_Stato ss ON o.id_ordine = ss.id_ordine
WHERE
    ss.stato = 'consegnato'
    AND DATE(ss.timestamp_modifica) = @giorno_riferimento;
```

Funzionalità 13

Lista dei membri del personale che hanno lavorato a un particolare ordine.

```
SELECT
    u.id_utente,
    u.nome AS membro_personale,
    u.email,
    ss.stato AS operazione_eseguita,
    ss.timestamp_modifica
FROM
    Storico_Stato ss
JOIN
    Utente u ON ss.id_utente_personale = u.id_utente
WHERE
    ss.id_ordine = 5
ORDER BY
    ss.timestamp_modifica ASC;
```

Funzionalità 14

Aggiornamento dello stato di un ordine.

```
INSERT INTO Storico_Stato (id_ordine, stato, id_utente_personale)
VALUES (5, 'in preparazione', 3);
```